

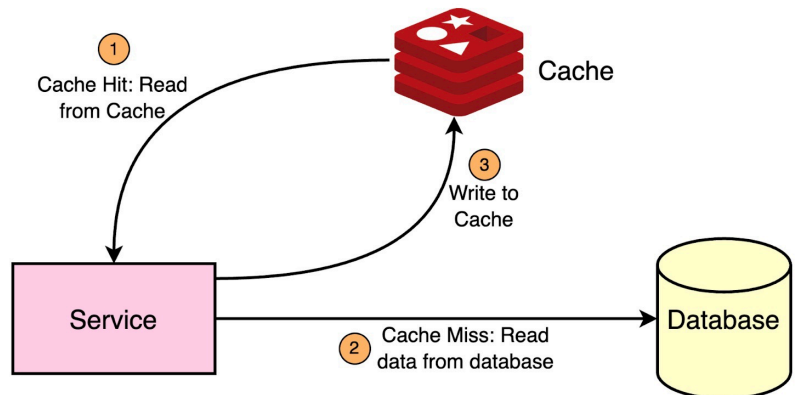
Caching Strategies

1. Cache-Aside (Lazy Loading) - Most Common

- Application **checks** the **cache**.
- If the **data is there**, **return** it.
- If **not**, **fetch** from the **database**, store it in the cache, and return it.

pros: only caches data when needed, which keeps the cache lean

cons: cache miss causes extra latency

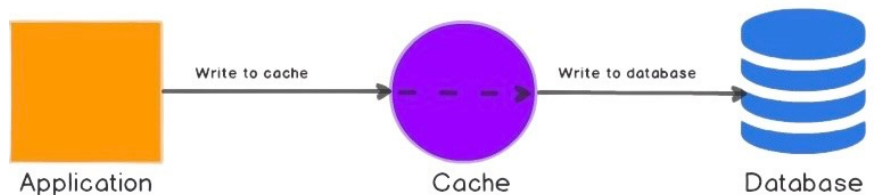


2. Write-Through Caching

- application **writes only to the cache**
- **cache** then **synchronously writes** to the **database** before returning to the application
- read happens from cache

pros: reads always return fresh data

cons: slower writes



3. Write-Back Caching

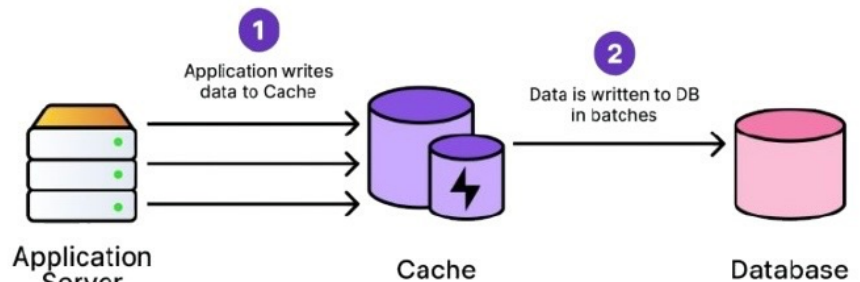
- application **writes only to the cache**.
- **cache batches** and **writes** the data to the **database asynchronously** in the background

pros:

makes writes very fast, great for high write throughput & eventual consistency applications

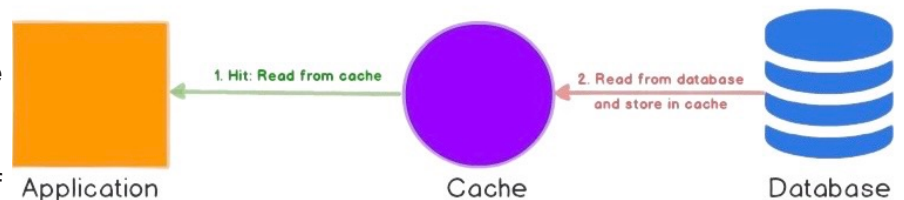
cons:

If the cache crashes before flushing, you can lose data



4. Read-Through Caching

- **read** always **happens** from **cache**
- application **never talks to the database** directly
- On a **cache miss**, the **cache** itself **fetches** from the **database**, **stores** the data, and returns it.



pros: Centralized logic

cons: Cache becomes complex